Program 5 Report

Title:
Verilog Coding for State Machines

Author:
Magnus Arnfinn Aubell

Date: October 16, 2025

Subject: EECE375

1 Purpose

In digital systems, transforming a signal from parallel to sequential is often necessary for sending digital data. This is the purpose of this program. The input will be a 8 bit parallel signal, $DATA$, and the output $SERIAL\_DATA\_OUT$ will be the same signal sent out serially with a frequency of 10Mbps. The leftmost and most significant bit will be the first sent out in the serial sequence. The system will assume that it receives an input clock signal CLK_{in} of 5Hz. For the first positive edge of the systems 10Hz clock, a 0 will always be sent out. The system will also have a active-low reset signal RST_N.

Further specifications can be found in [1], which the purpose section is largely based on.

2 Design Description

The design is split into 3 modules: a clock divider, a fsm and a parallel-to-serial converter which utlizes the first modules.

2.1 Clock Divider

The clock divider was designed using a simple Moore FSM that utilized the input clock frequency CLK_{in} and the active-low reset signal RST_N. The output CLK_{out} is decided by this FSM. The next state table for the FSM is shown in table 1. The full code for the module is shown in listing 2.

Table 1: Next-State Table for CLK_divider module

RST_N	CLK_out (current)	CLK_out (next)
0	0	0
0	1	0
1	0	1
1	1	0

Listing 1: Clock Divider by 2 with Active-Low Reset

```
//Clock divider made for parallel-to-serial converter
//Made by Magnus Arnfinn Aubell
//Last modified 30-09-2025

`timescale 1ns / 1ps


module CLK_divider(
    input CLK_in,
    input RST_N,
    output reg CLK_out
    );

    always @(posedge CLK_in or negedge RST_N) begin
        if (!RST_N) begin
            CLK_out <= 0;
        end else begin
            CLK_out <= !CLK_out;
        end
    end
endmodule
```

2.2 FSM

The input $DATA$ consists of 8 different bits. In addition to this the first positive edge of CLK_{out} is to be ignored. Lastly, after the whole $DATA$ signal has been sent, the system should send out 0. This in total makes up 10 total different states. The FSM therefore consists of a simple counter. For this FSM a Moore FSM was also chosen. Which bit that is being sent out for each state is shown in table 2. The code for the module is shown in listing **??**.

Table 2: Mapping of state to SERIAL_DATA_OUT

State (binary)	State (decimal)	SERIAL_DATA_OUT
0000	0	0
0001	1	DATA[7]
0010	2	DATA[6]
0011	3	DATA[5]
0100	4	DATA[4]
0101	5	DATA[3]
0110	6	DATA[2]
0111	7	DATA[1]
1000	8	DATA[0]
default	-	0

Listing 2: FSM module with 10 states.

```
//FSM counter with 10 states and reset function
//Made by Magnus Arnfinn Aubell
//Last modified 30-09-2025
//Note: Assumes CLK_in 5Hz in order to get CLK_out at 10Hz

`timescale 1ns / 1ps

module FSM(
    input RST_N,
    input CLK_in,
    input CLK_out,
    output reg [3:0] state
    );

     always @(posedge CLK_in or negedge RST_N) begin
        if (!RST_N) begin
            state <= 4'b0;
        end
        else if (CLK_out) begin
            if (state != 4'b1001) begin
                state <= state + 1;
            end
        end
     end

endmodule
```

Note that for the FSM CLK_{out} was implemented as if it were a variable, not as if it was a clock signal. This is due to errors from Verilog during synthesis when trying to use clock signal made in non-standard ways, like by a clock divider FSM.

2.3 Parallel-to-Serial Converter

The last part of the system is which utilizes the previously mentioned modules is the parallel-to-serial converter. The module code is shown in listing 3. The use of cases for the code was made following a lecture by Lee Sunggu given on 29-09-2025. [2]

Listing 3: Parallel-to-Serial Converter.

```
//Parallel-to-Serial Converter
//Made by Magnus Arnfinn Aubell
//Last modified 28-09-2025
//Case statement based on lecture given by Lee Sunggu:
//[2] Sunggu Lee. Computer design     eece375 lecture. In-person lecture, 2025. Lecture given 29 September 2025.
//Note: Assumes CLK_in 5Hz in order to get CLK_out at 10Hz

`timescale 1ns / 1ps


module SPI_MA(
    input CLK_in,
    input RST_N,
    input [7:0] DATA,
    output reg SERIAL_DATA_OUT,
    output CLK_out
    );


    wire [3:0] state;
    CLK_divider clk_divider (
        .CLK_in(CLK_in),
        .RST_N(RST_N),
        .CLK_out(CLK_out)
    );

    FSM fsm (
    .CLK_in(CLK_in),
    .CLK_out(CLK_out),
    .RST_N(RST_N),
    .state(state)
    );

    always @(posedge CLK_in or negedge RST_N) begin
        if (!RST_N) begin
        SERIAL_DATA_OUT <= 0;
    end else if (CLK_out) begin
        case(state)
            4'b0000 : SERIAL_DATA_OUT <= 0;
            4'b0001 : SERIAL_DATA_OUT <= DATA[7];
            4'b0010 : SERIAL_DATA_OUT <= DATA[6];
            4'b0011 : SERIAL_DATA_OUT <= DATA[5];
            4'b0100 : SERIAL_DATA_OUT <= DATA[4];
            4'b0101 : SERIAL_DATA_OUT <= DATA[3];
            4'b0110 : SERIAL_DATA_OUT <= DATA[2];
            4'b0111 : SERIAL_DATA_OUT <= DATA[1];
            4'b1000 : SERIAL_DATA_OUT <= DATA[0];
            default : SERIAL_DATA_OUT <= 0;
        endcase
    end
end

endmodule
```

2.4 Parallel-to-Serial Converter Testbench

Listing 4: Parallel-to-Serial Converter Testbench.

```
//Testbench for Parallel-to-Serial Converter
//Made by Magnus Arnfinn Aubell
//Last modified 28-09-2025

`timescale 1ns / 1ps

module tb_SPI_MA;
    reg CLK_in;
    reg RST_N;
    reg [7:0] DATA;
    wire SERIAL_DATA_OUT;
    wire CLK_out;

    SPI_MA spi (
        .CLK_in(CLK_in),
        .RST_N(RST_N),
        .DATA(DATA),
        .SERIAL_DATA_OUT(SERIAL_DATA_OUT),
        .CLK_out(CLK_out)
    );

    initial CLK_in = 0;
    always #100 CLK_in = !CLK_in;

    initial begin
        RST_N = 0;
        DATA  = 8'b10101010;
        #100;
        RST_N = 1;

        #5000;
        RST_N = 0;
        #5
        DATA  = 8'b10011101;
        #5;
        RST_N = 1;

        #10000
        $stop;
    end

    initial begin
        $monitor("Time=%0t | CLK_in=%b | RST_N=%b | DATA=%b | SERIAL_DATA_OUT=%
            b",
                 $time, CLK_in, RST_N, DATA, SERIAL_DATA_OUT);
    end

endmodule
```

3 Results

3.1 Waveforms

The circuit was tested for the $DATA$ input 1010_1010, 1001_1101 and 0010_0100. The corresponding waveforms to these tests are shown in figures 1, 2 and 3. These figures all stem from the same test/waveforms, but is split into different figures for convenience.

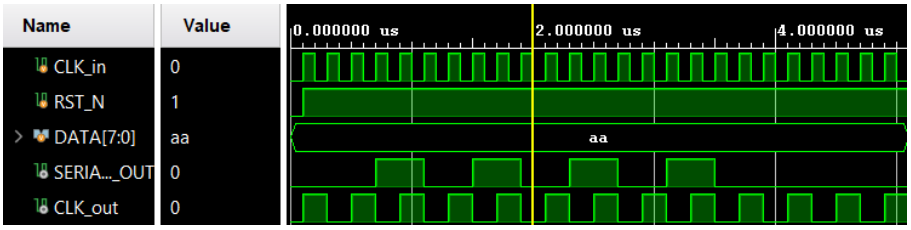

Figure 1: Response for input $DATA = 1010\_1010$.

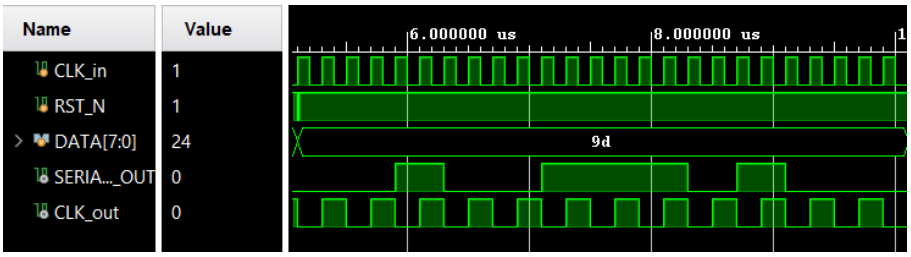

Figure 2: Response for input $DATA = 1001\_1101$.

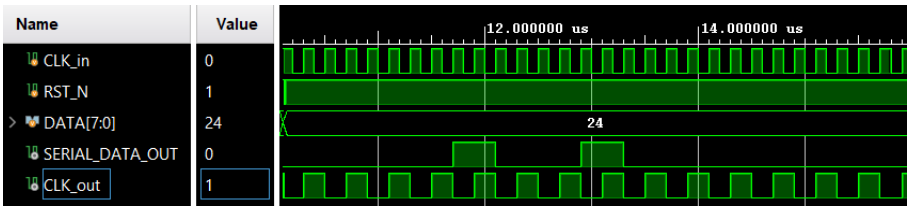

Figure 3: Response for input $DATA = 0010\_0100$.

3.2 Synthesis Diagram

The circuit was synthesized and redrawn using logic gates. The circuit was synthesized with "-flatten_hierarchy" setting set to "none". The redrawn circuit diagram is shown in figure 4.

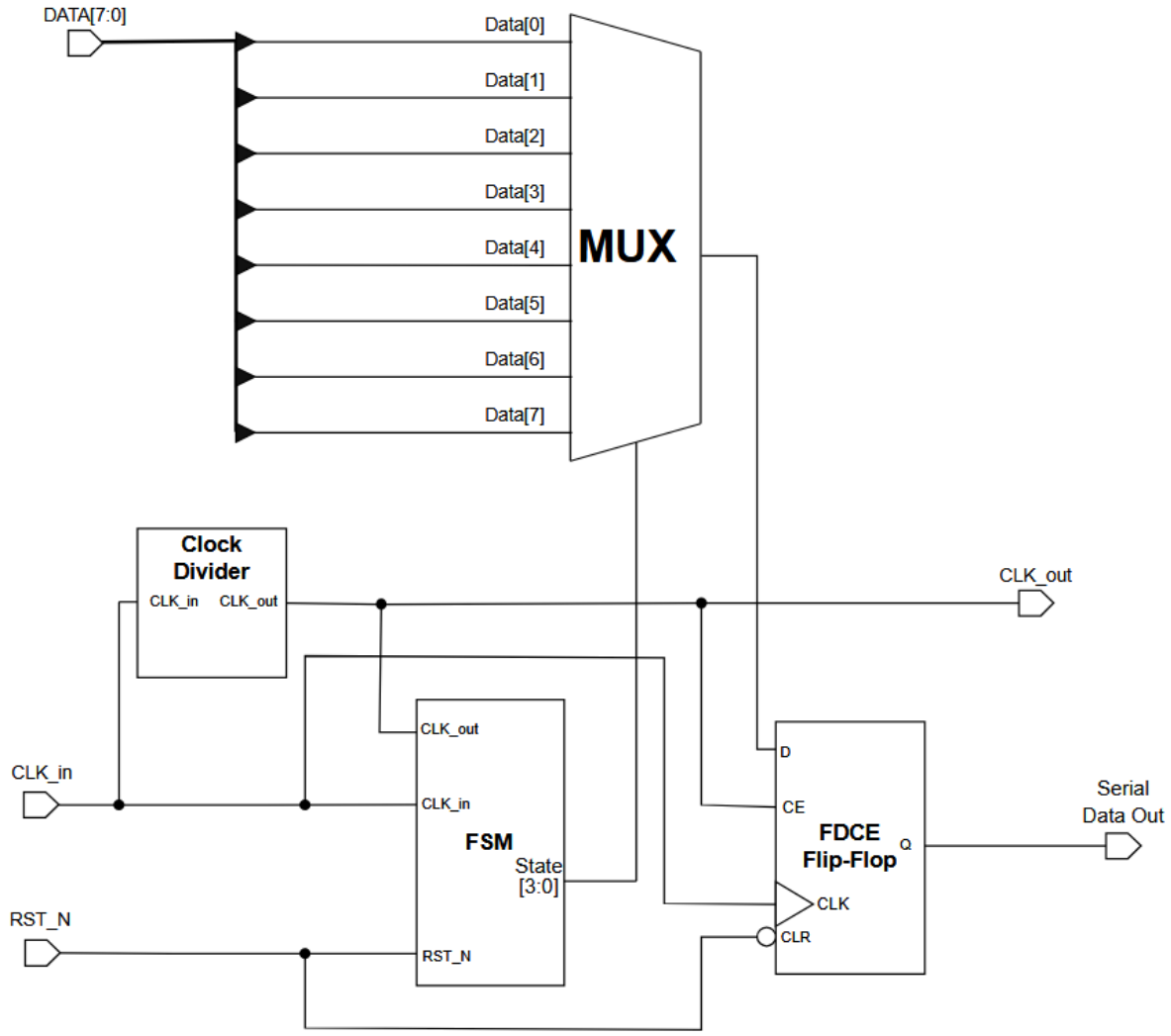

Figure 4: SPI circuit diagram.

Each module used in the SPI circuit's diagram in figure 4 is shown in figures 5-6.

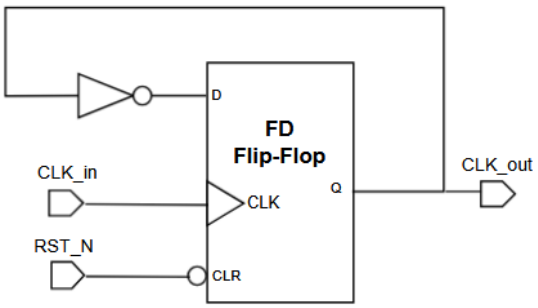

Figure 5: Clock divider diagram.

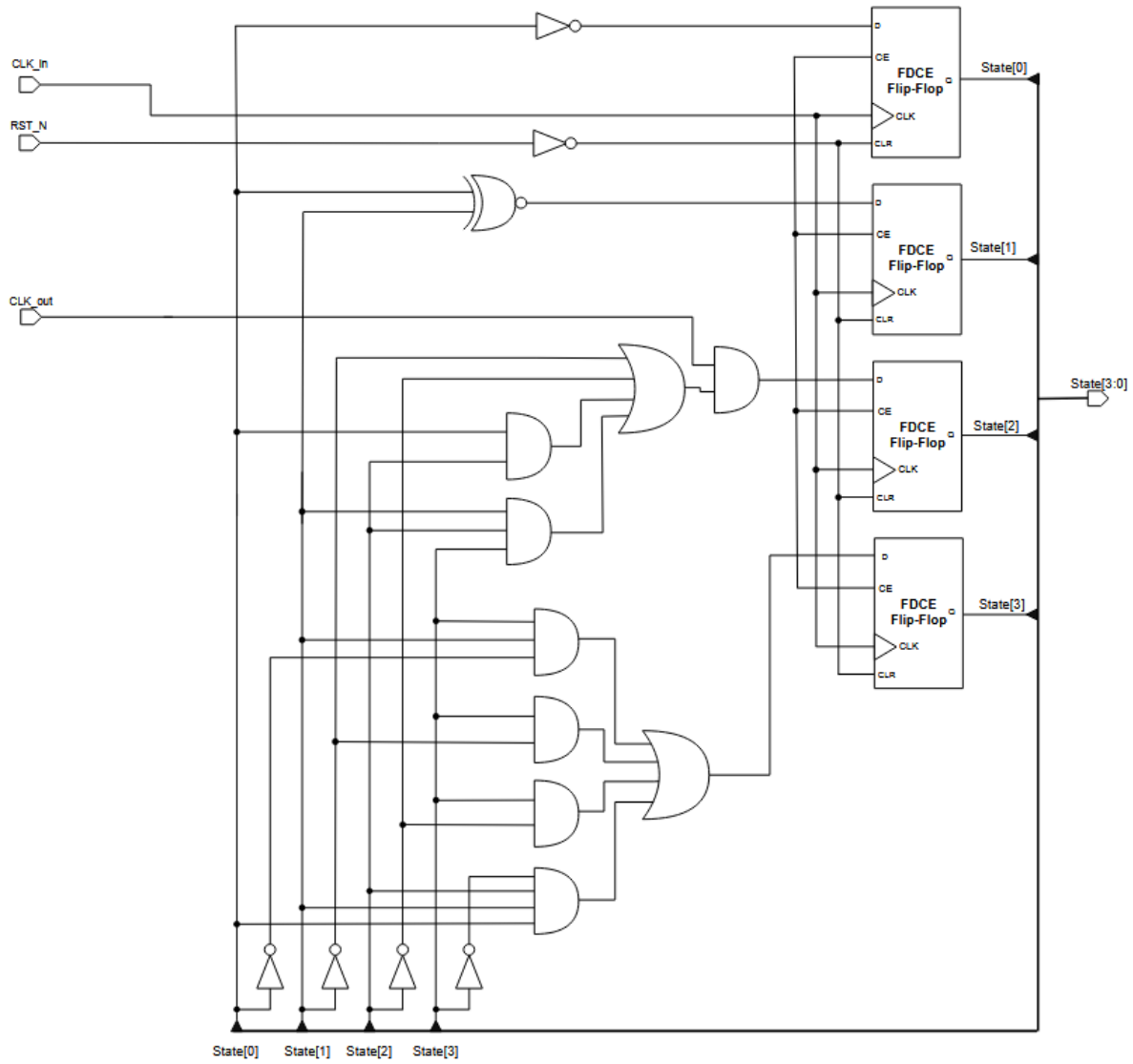

Figure 6: FSM Diagram

4 Discussion

The results were as expected. Originally all parts of the seriell-to-parallell coverter was done in one single module. This endede up giving quite a complex synthesis, even with "-flatten_hierarchy" set to "none". Therefore the module was split, in accordance to a discussion with the TA.

References

[1] Sung Gu Lee. Program 5: Verilog coding for state machines. EECE375, 2025. Accessed: 2025-09-30.

[2] Sunggu Lee. Computer design – eece375 lecture. In-person lecture, 2025. Lecture given 29 September 2025.